



گزارش جامع پروژه شماره ۳

طراحی و پیاده سازی سیستم طبقه بندی تصاویر زبان اشاره و مترجم
بلادرنگ به گفتار

نام درس

استاد

دانشجوی اول

دانشجوی دوم

مخزن پروژه

پروژه هوش مصنوعی

دکتر تن قطاری

فاطمه بهزادی دیل | شماره دانشجویی اول

فاطمه دمیرچی | ۴۰۳۱۱۰۲۴۱

github.com/FDamirchi/sign-language-translator

چکیده

این پروژه یک سامانه انتهابه‌انتهای تشخیص حروف زبان اشاره آمریکایی از تصویر، تجمیع خروجی به متن و تبدیل متن نهایی به گفتار را پیاده‌سازی می‌کند. داده مورد استفاده شامل ۸۷۰۰۰ تصویر آموزشی در ۲۹ کلاس است: حروف A-Z و سه فرمان `del`، `nothing` و `space`. تحلیل اکتشافی نشان می‌دهد تمام کلاس‌های مجموعه آموزشی دقیقاً ۳۰۰۰ تصویر دارند، تصاویر همگی RGB و با اندازه ثابت 200×200 پیکسل هستند و در نتیجه داده از نظر تعداد نمونه کاملاً متوازن است.

در فاز مدل‌سازی، یک شبکه عصبی پیچشی چهاربخشی با ۱۹۲۷۸۱۷۳ پارامتر طراحی و با چارچوب PyTorch آموزش داده شد. داده‌ها با نسبت‌های ۷۵/۱۵/۱۰ به آموزش، اعتبارسنجی و آزمون تقسیم شدند. مدل با بهینه‌ساز Adam، نرخ یادگیری 0.001، اندازه دسته 64 و به مدت 10 دوره آموزش دید. بهترین دقت اعتبارسنجی در دوره نهم برابر ۹۹.۲۳٪ و دقت آزمون نهایی برابر ۹۹.۳۴٪ گزارش شد.

در فاز بلادرنگ، یک معماری ماژولار برای دریافت تصویر وب‌کم، استخراج ناحیه دست، همسان‌سازی پیش‌پردازش با مدل، استنتاج روی CPU/GPU، تثبیت زمانی پیش‌بینی، مدیریت فرمان‌های ویرایشی، تشخیص پایان رشته و پخش صوت با pyttsx3 ایجاد شد. علامت تنها زمانی ثبت می‌شود که با اطمینان حداقل ۸۰٪ برای ثانیه ۲ پایدار بماند؛ کلاس NOTHING پس از ثانیه ۰.۴ قفل علامت قبلی را آزاد و پس از ثانیه ۵ متن موجود را نهایی می‌کند. کد با تست‌های واحد برای اجزای اصلی و کنترل کیفیت با pytest و Ruff همراه شده است.

سامانه حاضر یک مترجم حروف به حروف مبتنی بر `fingerspelling` است. بنابراین خروجی آن از توالی کلاس‌های ایستا ساخته می‌شود و جایگزین یک مدل کامل ترجمه زبان اشاره طبیعی پیوسته، شامل حرکات پویا و دستور زبان مستقل زبان اشاره، نیست.

فهرست مطالب

چکیده

آ

۱ تعریف مسئله، اهداف و معیارهای تحویل

۱.۱ بیان مسئله

۲.۱ اهداف فنی

۳.۱ تطبیق الزامات صورت پروژه با خروجی نهایی

۲ فاز اول: تحلیل اکتشافی و پیش‌پردازش داده

۱.۲ ساختار مجموعه داده

۲.۲ تصویرسازی نمونه‌ها

۳.۲ ویژگی‌های تصاویر

۴.۲ تقسیم‌بندی داده

۵.۲ نرمال‌سازی

۳ فاز دوم: طراحی و آموزش شبکه عصبی پیچشی

۱.۳ معماری مدل

۲.۳ افزایش داده و کنترل بیش‌برازش

۳.۳ پارامترهای آموزش

۴.۳ روند آموزش

۵.۳ ارزیابی نهایی

۴ فاز سوم: سامانه بلادرنگ ترجمه به متن و گفتار

۱۳

۱.۴	معماری کلان	۱۳
۲.۴	ساختار ماژول‌ها	۱۳
۳.۴	دریافت تصویر و ناحیه مورد توجه	۱۴
۴.۴	همسان‌سازی پیش‌پردازش آموزش و اجرا	۱۴
۵.۴	بارگذاری مدل و استنتاج	۱۵
۶.۴	منطق تثبیت زمانی	۱۵
۷.۴	تفسیر برجسب‌ها و بافر متن	۱۶
۸.۴	تشخیص پایان عبارت و تبدیل به گفتار	۱۶
۹.۴	رابط تصویری	۱۷
۵	اعتبارسنجی نرم‌افزار و کیفیت پیاده‌سازی	۱۸
۱.۵	مجموعه تست‌ها	۱۸
۲.۵	مدیریت خطا	۱۹
۳.۵	قابلیت تعویض اجزا	۱۹
۶	تحلیل محدودیت‌ها و پیشنهادهای بهبود	۲۰
۱.۶	شکاف دامنه میان دیتاست و وب‌کم	۲۰
۲.۶	حروف و حرکات پویا	۲۰
۷	راهنمای اجرا و ساختار تحویل	۲۱
۱.۷	نصب	۲۱
۲.۷	اجرای کنترل کیفیت	۲۱
۳.۷	اجرای سامانه	۲۱
۴.۷	ساختار فایل نهایی تحویل	۲۲
۸	نتیجه‌گیری	۲۳
آ	تنظیمات نهایی سامانه	۲۴
ب	قرارداد وزن‌های مدل	۲۵

فهرست تصاویر

۱.۲	توزیع کلاس‌ها در پوشه آموزشی و آزمون اولیه؛ خروجی مستقیم نوت‌بوک فاز ۱	۴
۲.۲	نمونه‌ای از هر کلاس مجموعه آموزشی؛ تمام تصاویر 200×200 RGB	۵
۳.۲	نسبت بخش‌های آموزش، اعتبارسنجی و آزمون	۷
۱.۳	معماری شبکه SignLanguageCNN	۸
۲.۳	دقت آموزش و اعتبارسنجی؛ بازسازی شده از خروجی‌های متنی نوت‌بوک	۱۱
۱.۴	زنجیره پردازش بلادرنگ از وب‌کم تا گفتار	۱۳
۲.۴	ماشین حالت رمزگشایی زمانی	۱۶

فهرست جداول

۱.۱	پوشش الزامات اصلی پروژه	۲
۱.۲	خلاصه ویژگی‌های تصویری داده	۶
۲.۲	تقسیم‌بندی نهایی داده برای مدل‌سازی	۶
۱.۳	جزئیات لایه‌ها و تعداد پارامترهای قابل آموزش	۹
۲.۳	ابزارهای آموزش	۱۰
۳.۳	دقت ثبت‌شده در هر دوره	۱۱
۴.۳	نتایج نهایی مدل	۱۲
۱.۴	رفتار هر نوع برچسب پذیرفته‌شده	۱۶
۱.۵	حوزه‌های تحت آزمون واحد	۱۸
۱.آ	مقادیر پیش‌فرض تنظیمات اجرایی	۲۴

تعریف مسئله، اهداف و معیارهای تحویل

۱.۱ بیان مسئله

هدف پروژه توسعه یک سیستم کامل هوش مصنوعی از مرحله شناخت داده خام تا استقرار در یک محیط کاربردی بلادرنگ است. ورودی سیستم فریم‌های دوربین و خروجی آن متن حروف‌چینی‌شده و صوت متن نهایی است. مسئله اصلی را می‌توان به سه زیرمسئله تقسیم کرد:

۱. تحلیل و استانداردسازی تصاویر کلاس‌بندی‌شده زبان اشاره؛
۲. آموزش یک طبقه‌بند CNN برای تشخیص یکی از ۲۹ کلاس؛
۳. طراحی منطق زمانی برای تبدیل پیش‌بینی‌های فریم‌به‌فریم به رشته پایدار و قابل خواندن.

۲.۱ اهداف فنی

- ▶ بررسی توازن کلاس‌ها، ویژگی‌های تصویر و توزیع داده؛
- ▶ طراحی شبکه شامل لایه‌های Convolution، MaxPooling و Fully Connected؛
- ▶ کاهش بیش‌برازش با افزایش داده، Dropout و تنظیم نرخ یادگیری؛
- ▶ دریافت تصویر با OpenCV و محدودسازی ورودی مدل به یک ناحیه مربعی؛
- ▶ ثبت علامت تنها پس از تشخیص پایدار برای جلوگیری از نویز لحظه‌ای؛
- ▶ پشتیبانی از فرمان‌های DEL، SPACE و NOTHING؛
- ▶ تبدیل رشته نهایی به گفتار و آماده‌سازی سیستم برای عبارت بعدی؛
- ▶ جداسازی اجزا و فراهم کردن تست‌های واحد برای توسعه و نگهداری مطمئن.

۳.۱ تطبیق الزامات صورت پروژه با خروجی نهایی

جدول ۱.۱: پوشش الزامات اصلی پروژه

فاز	الزام	وضعیت
فاز ۱	شمارش نمونه‌ها و بررسی توازن ۲۹ کلاس	انجام‌شده
فاز ۱	نمایش نمونه تصادفی از هر کلاس در شبکه تصویری	انجام‌شده
فاز ۱	تحلیل ابعاد، کانال رنگ و بازه پیکسل‌ها	انجام‌شده
فاز ۱	تقسیم داده به آموزش، اعتبارسنجی و آزمون	انجام‌شده
فاز ۲	طراحی و آموزش CNN و شرح معماری	انجام‌شده
فاز ۲	بررسی اثر پارامترهایی نظیر نرخ یادگیری، اندازه دسته و بهینه‌ساز	انجام‌شده
فاز ۲	ارزیابی روی داده آزمون و گزارش دقت	انجام‌شده
فاز ۲	نمودار Accuracy و Loss و ماتریس درهم‌ریختگی	انجام‌شده
فاز ۳	دریافت تصویر وب‌کم، کادر دست و ارسال ROI به مدل	انجام‌شده
فاز ۳	تثبیت علامت برای ۲ ثانیه با اطمینان بالا	انجام‌شده
فاز ۳	نهایی‌سازی بعد از ۵ ثانیه NOTHING	انجام‌شده
فاز ۳	تبدیل متن به گفتار و پاک‌کردن رشته	انجام‌شده

کد آموزش میانگین loss را محاسبه می‌کند و زمان‌بند نرخ یادگیری را بر مبنای validation loss به‌روزرسانی می‌کند، اما مقادیر تاریخی train loss و validation loss در خروجی نوت‌بوک ذخیره یا رسم نشده‌اند. همچنین وجود شکل نهایی ماتریس درهم‌ریختگی در خروجی متعهدشده نوت‌بوک قابل تأیید نیست. چون صورت پروژه این دو خروجی را صریحاً الزامی کرده است، پیش از تحویل باید نوت‌بوک یک‌بار با ذخیره تاریخچه زیان و رسم ماتریس درهم‌ریختگی اجرا شود. در این گزارش هیچ مقدار زیان تخمین زده نشده است.

فاز اول: تحلیل اکتشافی و پیش‌پردازش داده

۱.۲ ساختار مجموعه داده

مجموعه آموزشی شامل ۲۹ پوشه کلاس است. ۲۶ کلاس نخست حروف الفبای انگلیسی و سه کلاس آخر برای کنترل رشته در محیط بلادرنگ در نظر گرفته شده‌اند:

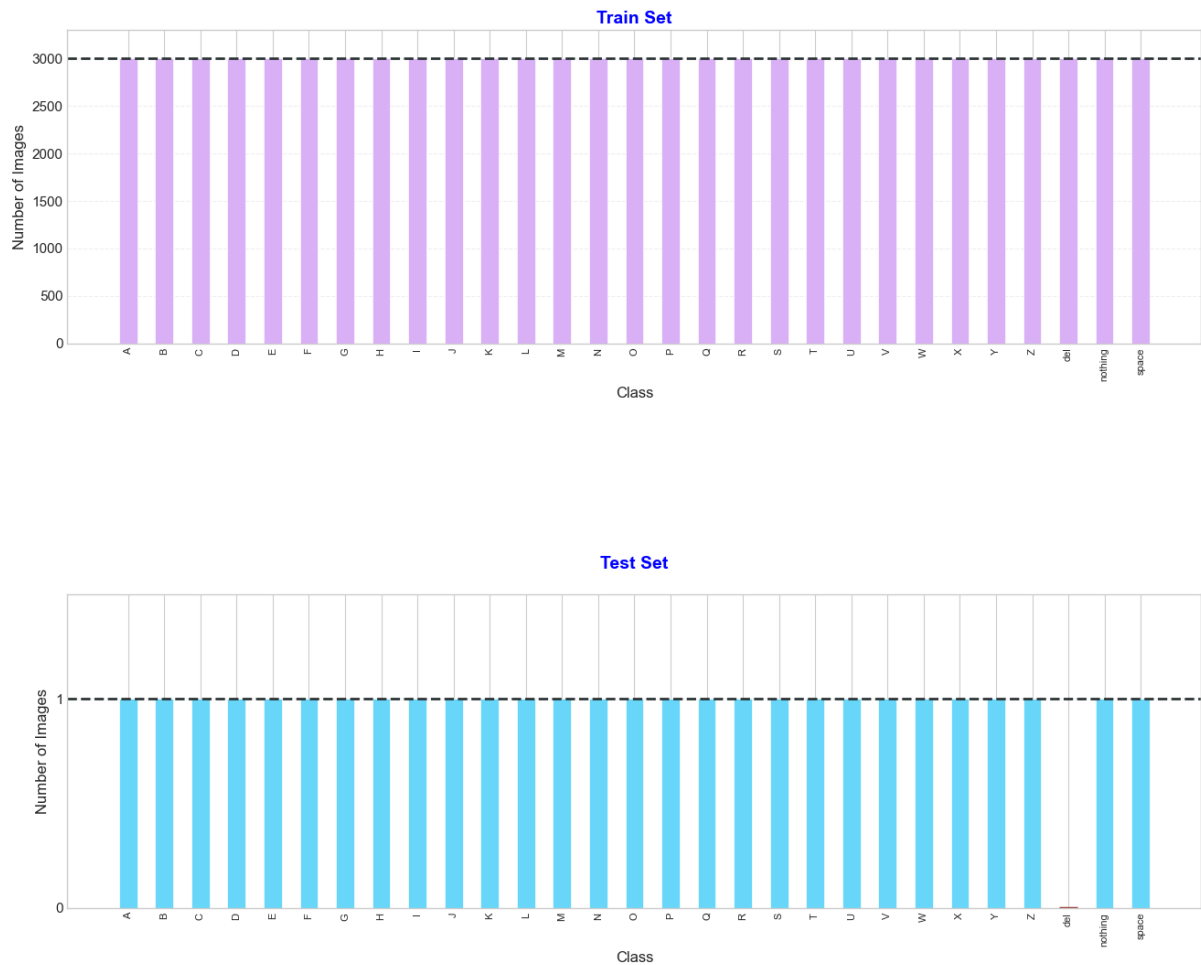
A, B, C, ..., Z, del, nothing, space

هر کلاس دقیقاً ۳۰۰۰ تصویر دارد؛ بنابراین تعداد کل تصاویر آموزشی برابر است با:

$$29 \times 3000 = 87000$$

در پوشه آزمون اولیه دیتاست تنها ۲۸ تصویر، هر کلاس یک تصویر، وجود داشته و کلاس del غایب بوده است. این مجموعه آزمون کوچک برای ارزیابی آماری مدل کافی نبود؛ از این رو برای فاز آموزش یک تقسیم‌بندی جدید از تصاویر اصلی ایجاد شد.

Class Distribution: Train & Test

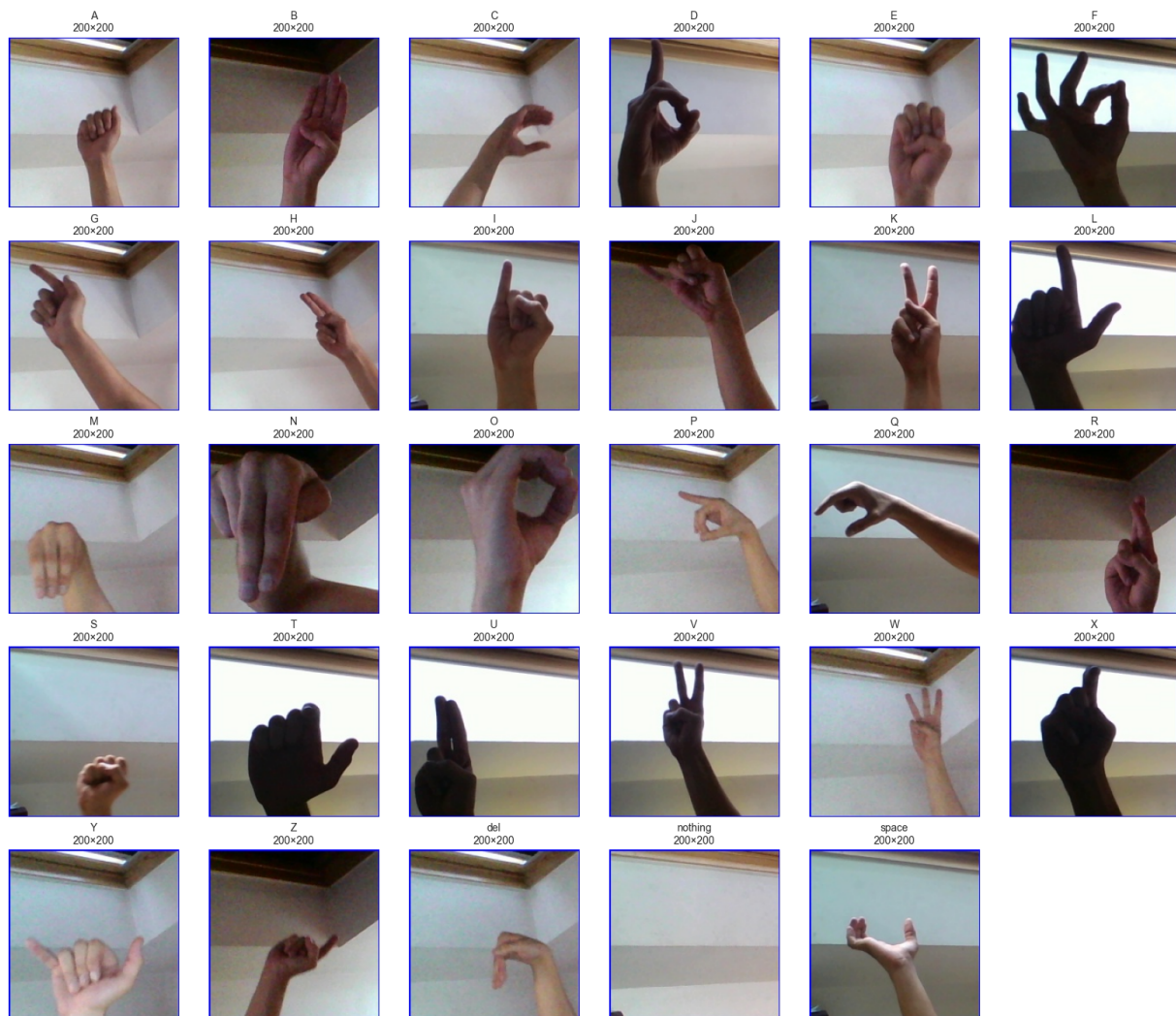


شکل ۱.۲: توزیع کلاس‌ها در پوشه آموزشی و آزمون اولیه؛ خروجی مستقیم نوت‌بوک فاز ۱

۲.۲ تصویرسازی نمونه‌ها

شکل ۲.۲ یک تصویر تصادفی از هر یک از ۲۹ کلاس را نشان می‌دهد. تصاویر شامل دست، مچ و بخشی از ساعد هستند و پس‌زمینه ثابت و قابل‌توجهی نیز در آن‌ها دیده می‌شود. این ویژگی در زمان استقرار مهم است؛ زیرا یک شبکه کانولوشنی ممکن است علاوه بر شکل دست، از نور، زاویه و الگوی پس‌زمینه نیز سرنخ بگیرد.

Sample Image from Each Class In Train Set



شکل ۲.۲: نمونه‌ای از هر کلاس مجموعه آموزشی؛ تمام تصاویر 200×200 RGB

۳.۲ ویژگی‌های تصاویر

تحلیل تمام ۸۷ ۰۰۰ تصویر نتایج زیر را نشان داد:

جدول ۱.۲: خلاصه ویژگی‌های تصویری داده

ویژگی	نتیجه
حالت رنگ	همه تصاویر RGB
عرض	حداقل، حداکثر و میانگین همگی 200 پیکسل
ارتفاع	حداقل، حداکثر و میانگین همگی 200 پیکسل
نسبت تصویر	ثابت و برابر 1.00
بازه ذخیره‌سازی پیکسل	اعداد صحیح 0-255
توازن کلاس‌ها	انحراف معیار تعداد نمونه‌ها برابر صفر

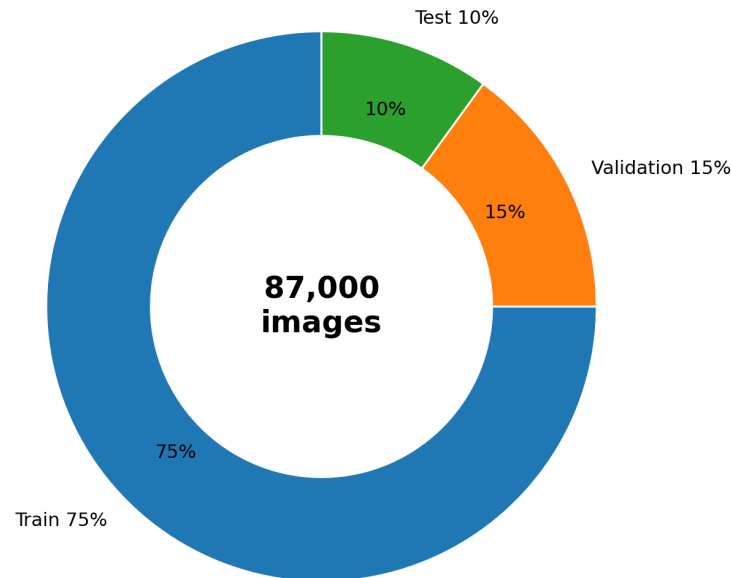
با وجود ثابت‌بودن اندازه داده، در هر دو مسیر آموزش و استنتاج عمل تغییر اندازه به 200×200 تکرار شده است تا ورودی‌های خارج از دیتاست نیز به شکل مورد انتظار مدل تبدیل شوند.

۴.۲ تقسیم‌بندی داده

از کل ۸۷ ۰۰۰ تصویر، تقسیم زیر استفاده شده است:

جدول ۲.۲: تقسیم‌بندی نهایی داده برای مدل‌سازی

بخش	تعداد تصویر	نسبت
Train	65,250	75%
Validation	13,050	15%
Test	8,700	10%
جمع	87,000	100%



شکل ۳.۲: نسبت بخش‌های آموزش، اعتبارسنجی و آزمون

۵.۲ نرمال‌سازی

برای هر کانال، میانگین و انحراف معیار روی تصاویر آموزش محاسبه شد:

$$\mu = (0.5188341, 0.4989899, 0.5144102),$$

$$\sigma = (0.2045820, 0.2336411, 0.2412035).$$

پس از تبدیل پیکسل‌ها به بازه $[0,1]$ ، هر کانال طبق رابطه زیر استاندارد شد:

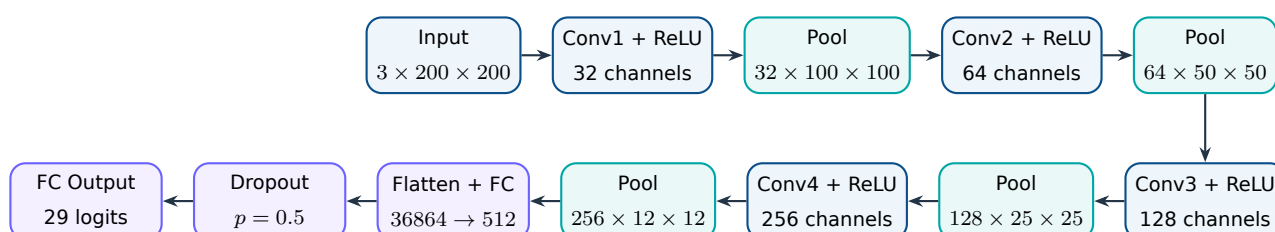
$$x'_c = \frac{x_c - \mu_c}{\sigma_c}$$

آمار محاسبه‌شده در فایل `dataset_stats.pt` ذخیره و عیناً در تنظیمات استنتاج بلادرنگ استفاده شده است. این همسانی برای جلوگیری از تغییر توزیع ورودی میان آموزش و اجرا ضروری است.

فاز دوم: طراحی و آموزش شبکه عصبی پیچشی

۱.۳ معماری مدل

مدل SignLanguageCNN چهار بلوک کانولوشن و بیشینه‌گیری، یک لایه پنهان تمام‌متصل و یک لایه خروجی ۲۹ کلاسه دارد. در تمام لایه‌های کانولوشن از هسته 3×3 ، $\text{padding}=1$ و تابع فعال‌ساز ReLU استفاده شده است. هر بلوک با $\text{MaxPool } 2 \times 2$ ابعاد مکانی را نصف می‌کند.



شکل ۱.۳: معماری شبکه SignLanguageCNN

جدول ۱.۳: جزئیات لایه‌ها و تعداد پارامترهای قابل آموزش

لایه	ورودی	خروجی پس از Pool	پارامتر	توضیح
Conv1	$3 \times 200 \times 200$	$32 \times 100 \times 100$	896	استخراج لبه‌ها و الگوهای ابتدایی
Conv2	$32 \times 100 \times 100$	$64 \times 50 \times 50$	18,496	ترکیب ویژگی‌های سطح پایین
Conv3	$64 \times 50 \times 50$	$128 \times 25 \times 25$	73,856	یادگیری الگوهای پیچیده‌تر شکل دست
Conv4	$128 \times 25 \times 25$	$256 \times 12 \times 12$	295,168	تولید نمایش فشرده سطح بالا
FC1	36,864	512	18,874,880	تبدیل ویژگی‌های مکانی به نمایش طبقه‌بندی
FC2	512	29	14,877	تولید لاجیت هر کلاس
مجموع			19,278,173	

بخش عمده پارامترهای مدل در لایه FC1 قرار دارد. این انتخاب ظرفیت طبقه‌بندی بالایی ایجاد می‌کند، اما هزینه حافظه و ریسک بیش‌برازش را نیز افزایش می‌دهد. وجود Dropout و افزایش داده برای کنترل این ریسک اهمیت دارد.

۲.۳ افزایش داده و کنترل بیش‌برازش

تبدیلات مسیر آموزش عبارت‌اند از:

► تغییر اندازه به 200×200 ؛

► چرخش تصادفی تا $\pm 10^\circ$ ؛

► وارونگی افقی تصادفی؛

► تغییر جزئی روشنایی و کنتراست با شدت 0.1؛

► تبدیل به تنسور و نرمال‌سازی با آمار داده آموزش.

برای اعتبارسنجی و آزمون تنها تغییر اندازه، تبدیل به تنسور و نرمال‌سازی انجام می‌شود تا معیارها روی داده بدون اغتشاش تصادفی محاسبه شوند.

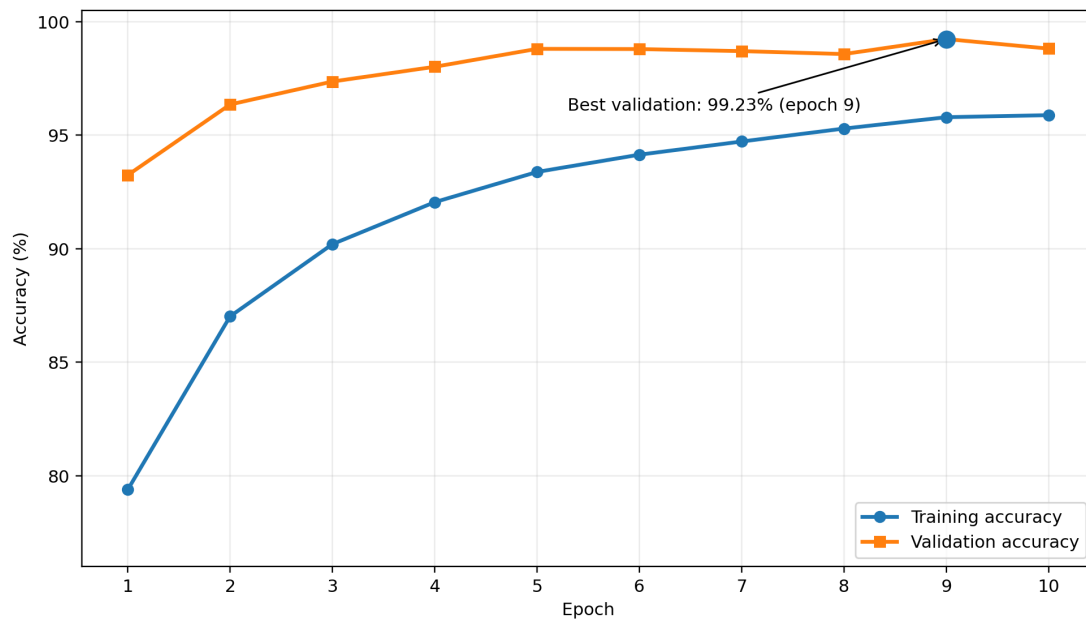
در معماری نهایی دو مورد از پیشنهادهای بخش امتیازی صورت پروژه پیاده‌سازی شده است: Data Augmentation و Dropout. همچنین ReduceLROnPlateau برای کاهش نرخ یادگیری در صورت توقف بهبود زبان اعتبارسنجی به کار رفته است. Batch Normalization و اتصالات میانبر در نسخه فعلی استفاده نشده‌اند.

۳.۳ پارامترهای آموزش

جدول ۳.۳: ابرپارامترهای آموزش

پارامتر	مقدار
تعداد دوره	10
اندازه دسته	64
نرخ یادگیری اولیه	0.001
بهینه‌ساز	Adam
تابع زیان	CrossEntropyLoss
زمان‌بند نرخ یادگیری	ReduceLROnPlateau
تنظیم زمان‌بند	patience=3, factor=0.5
معیار ذخیره بهترین مدل	بیشترین دقت اعتبارسنجی
فایل وزن‌ها	models/final_model.pth

۴.۳ روند آموزش



شکل ۴.۳: دقت آموزش و اعتبارسنجی؛ بازسازی شده از خروجی های متنی نوت بوک

جدول ۴.۳: دقت ثبت شده در هر دوره

دوره آموزش اعتبارسنجی	دوره آموزش	اعتبارسنجی	دوره آموزش اعتبارسنجی	دوره آموزش	اعتبارسنجی
۱	79.39%	93.24%	۶	94.14%	98.79%
۲	87.02%	96.35%	۷	94.72%	98.70%
۳	90.20%	97.36%	۸	95.29%	98.57%
۴	92.05%	98.01%	۹	95.79%	99.23%
۵	93.38%	98.80%	۱۰	95.88%	98.81%

بالا تر بودن دقت اعتبارسنجی نسبت به دقت آموزش الزاماً نشانه خطا نیست. در زمان آموزش، تبدیلات تصادفی و Dropout مسئله را سخت تر می کنند؛ در حالی که هنگام اعتبارسنجی شبکه در حالت eval قرار دارد و ورودی ها بدون افزایش داده اند. بهترین مدل در دوره نهم ذخیره شده است.

۵.۳ ارزیابی نهایی

نتایج ثبت شده روی مجموعه آزمون ۷۰۰ ۸ تصویری عبارت اند از:

جدول ۴.۳: نتایج نهایی مدل

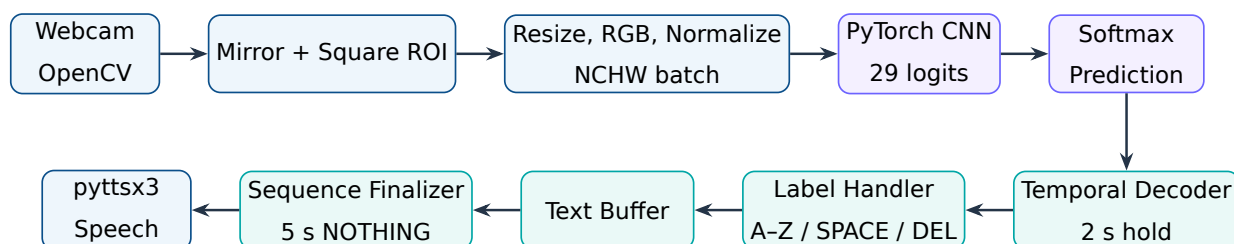
مقدار	معیار
8,643 / 8,700	پیش‌بینی صحیح
99.34%	دقت آزمون
98.98%	میانگین اطمینان
38.06%	کمینه اطمینان مشاهده‌شده
99.23%	بهترین دقت اعتبارسنجی

دقت بالای آزمون نشان می‌دهد مدل الگوهای موجود در منبع داده را به‌خوبی فراگرفته است. با این حال، تقسیم آموزش/اعتبارسنجی/آزمون از تصاویر یک مجموعه همگن انجام شده است؛ بنابراین این عدد به‌تنهایی تضمین‌کننده عملکرد برابر در نور، دوربین، پس‌زمینه و دست افراد جدید نیست. آزمون بلادرنگ وب‌کم باید به‌عنوان ارزیابی جداگانه دامنه واقعی در نظر گرفته شود.

فاز سوم: سامانه بلادرنگ ترجمه به متن و گفتار

۱.۴ معماری کلان

کد فاز سوم به صورت یک بسته ماژولار با چهار حوزه اصلی طراحی شده است: بینایی ماشین، استنتاج مدل، رمزگشایی زمانی و گفتار. جریان کامل داده در شکل ۱.۴ آمده است.



شکل ۱.۴: زنجیره پردازش بلادرنگ از وب کم تا گفتار

۲.۴ ساختار ماژول ها

```

src/sign_translator/
|-- app.py           # orchestration and main loop
|-- config.py        # camera/model/decoder/TTS parameters
|-- labels.py        # canonical label order and commands
|-- vision/
|   |-- camera.py    # safe camera lifecycle
|   |-- roi.py       # square region of interest
|   |-- overlay.py   # prediction and progress UI
|-- inference/
|   |-- preprocessing.py # BGR -> normalized NCHW
|   |-- torch_model.py  # SignLanguageCNN architecture

```

```
| |-- torch_predictor.py # weight loading and inference
| |-- output_decoder.py # logits/probabilities validation
| `-- factory.py          # mock or torch backend
|-- decoding/
| |-- temporal_decoder.py
| |-- sequence_finalizer.py
| |-- label_handler.py
| `-- text_buffer.py
`-- speech/
    |-- factory.py
    |-- noop.py
    `-- pytttsx3_engine.py
```

این جداسازی باعث می‌شود مدل، موتور گفتار و منطق رابط بدون بازنویسی حلقه اصلی قابل تعویض باشند. برای مثال، TimedMockPredictor پیش از تحویل مدل واقعی امکان تست کل سامانه را فراهم کرده و سپس TorchPredictor از طریق کارخانه جایگزین آن شده است.

۳.۴ دریافت تصویر و ناحیه مورد توجه

کلاس Camera دوربین را به صورت context manager باز و در پایان آزاد می‌کند. رزولوشن در تنظیمات روی 1280×720 قرار گرفته و تصویر برای تجربه طبیعی کاربر به صورت افقی آینه می‌شود. ناحیه دست با نسبت‌های مستقل از رزولوشن تعریف شده است:

$$\begin{aligned}x &= 0.55W, & y &= 0.12H, \\w &= 0.38W, & h &= 0.72H.\end{aligned}$$

سپس ضلع کوچک‌تر انتخاب و کادر دقیقاً مربعی می‌شود. این اقدام از فشردگی یا کشیدگی شکل دست هنگام تبدیل به ورودی 200×200 جلوگیری می‌کند.

۴.۴ همسان‌سازی پیش‌پردازش آموزش و اجرا

فریم OpenCV در قالب BGR است، در حالی که داده آموزشی RGB بوده است. مسیر استنتاج مراحل زیر را انجام می‌دهد:

۱. اعتبارسنجی ورودی سه کاناله و غیرخالی

۲. تغییر اندازه به 200×200

۳. تبدیل BGR به RGB

۴. تبدیل نوع به float32 و تقسیم بر 255

۵. نرمال سازی کانالی با میانگین و انحراف معیار آموزش

۶. تبدیل چیدمان از HWC به CHW

۷. افزودن بعد دسته و تولید تنسور $1 \times 3 \times 200 \times 200$

انتخاب روش درون یابی نیز بر اساس کوچک سازی یا بزرگ سازی انجام می شود: INTER_AREA برای کاهش اندازه و INTER_LINEAR برای افزایش اندازه.

۵.۴ بارگذاری مدل و استنتاج

TorchPredictor معماری را با تعداد خروجی برابر طول فهرست برچسب ها می سازد و فایل وزن را با `weights_` `only=True` بارگذاری می کند. استفاده از `strict=True` در `load_state_dict` سبب می شود هر ناسازگاری میان معماری و وزن ها به جای تولید نتیجه اشتباه، با خطای واضح متوقف شود. ترتیب خروجی مدل در کد به صورت زیر ثابت شده است:

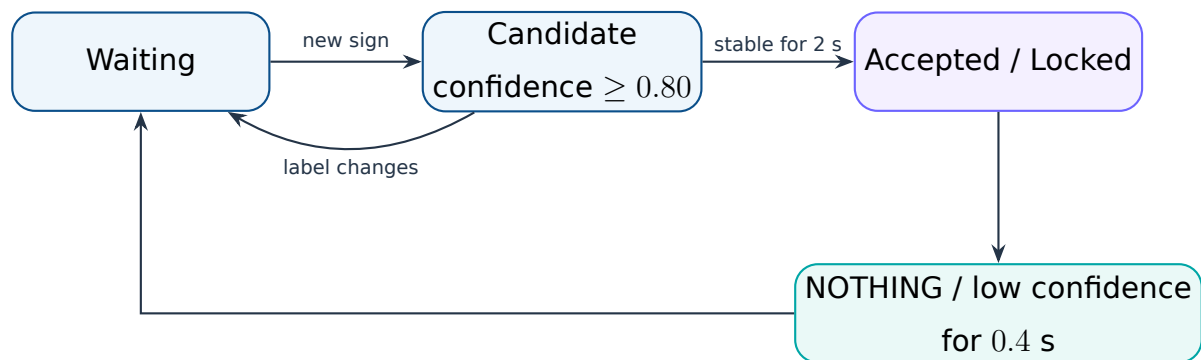
(A, ..., Z, DEL, NOTHING, SPACE)

خروجی شبکه لاجیت است. `output_decoder.py` شکل خروجی، تعداد کلاس ها، مقادیر نامتناهی و تکراری نبودن برچسب ها را بررسی و سپس softmax پایدار عددی را محاسبه می کند.

۶.۴ منطق تثبیت زمانی

پیش بینی خام هر فریم مستقیماً وارد متن نمی شود. TemporalDecoder یک ماشین حالت ساده دارد:

- ▶ پیش بینی باید اطمینان حداقل 0.80 داشته باشد
- ▶ یک برچسب باید به مدت 2.0 s بدون تغییر باقی بماند
- ▶ پس از ثبت، همان برچسب قفل می شود تا با نگه داشتن دست چندبار تکرار نشود
- ▶ نمایش NOTHING یا افت اطمینان به مدت 0.4 s قفل را آزاد می کند
- ▶ برای تکرار دو حرف یکسان، مانند OO، کاربر باید بین دو نمایش، حالت NOTHING را نشان دهد



شکل ۲.۴: ماشین حالت رمزگشایی زمانی

۲.۴ تفسیر برچسب‌ها و بافر متن

جدول ۱.۴: رفتار هر نوع برچسب پذیرفته‌شده

برچسب	عملکرد
A-Z	افزودن حرف انگلیسی بزرگ به انتهای بافر
SPACE	افزودن یک فاصله؛ فاصله ابتدایی یا تکراری نادیده گرفته می‌شود
DEL	حذف آخرین کاراکتر، شامل حرف یا فاصله
NOTHING	عدم تغییر متن؛ استفاده به عنوان علامت خنثی و پایان رشته

TextBuffer به جای ذخیره توکن‌های کلمه، فهرست کاراکترها را نگه می‌دارد. بنابراین فرمان حذف دقیقاً آخرین کاراکتر را برمی‌گرداند و متن نهایی با حذف فاصله‌های ابتدا و انتها تولید می‌شود.

۸.۴ تشخیص پایان عبارت و تبدیل به گفتار

SequenceFinalizer تنها زمانی شمارنده را فعال می‌کند که:

► برچسب NOTHING با اطمینان حداقل 0.80 تشخیص داده شود.

► بافر متن خالی نباشد.

► این حالت برای 5.0 s پیوسته ادامه یابد.

پس از تکمیل شمارنده، متن یکبار نهایی می‌شود، به موتور گفتار ارسال و سپس بافر پاک می‌شود. پرچم داخلی از چندبارخواندن متن در همان دوره طولانی NOTHING جلوگیری می‌کند.

موتور pyttsx3 در یک نخ کارگر اجرا می شود و متن ها را از صف دریافت می کند. این طراحی مانع توقف حلقه پردازش ویدئو هنگام پخش صدا می شود. یک پیاده سازی NoOp نیز برای تست و محیط های فاقد صوت وجود دارد.

۹.۴ رابط تصویری

overlay.py اطلاعات زیر را روی فریم نشان می دهد:

- ▶ کادر سبز ناحیه دست
- ▶ برچسب پیش بینی شده و درصد اطمینان
- ▶ درصد پیشرفت نگه داشتن علامت
- ▶ وضعیت قفل، پذیرش یا آزاد شدن علامت
- ▶ درصد پیشرفت نهایی سازی متن
- ▶ متن جاری و آخرین متن نهایی شده
- ▶ راهنمای خروج با کلیدهای Q یا ESC

اعتبارسنجی نرم افزار و کیفیت پیاده سازی

۱.۵ مجموعه تست ها

پوشه tests اجزای زیر را پوشش می دهد:

جدول ۱.۵: حوزه های تحت آزمون واحد

فایل تست	رفتارهای بررسی شده
test_roi.py	محاسبه کادر، برش تصویر و مربعی بودن ناحیه ورودی
test_preprocessing.py	شکل NCHW، تبدیل رنگ، نوع داده و نرمال سازی کانال ها
test_torch_model.py	تولید ۲۹ لاجیت برای دسته ورودی 200×200
test_torch_predictor.py	اتصال پیش پردازش به مدل ساختگی، نگاشت خروجی و مدیریت فایل مفقود
test_output_decoder.py	لاجیت، احتمال، softmax، تعداد کلاس و مقادیر نامعتبر
test_temporal_decoder.py	شرط دوثانیه ای، جلوگیری از تکرار، آزاد شدن و آستانه اطمینان
test_sequence_finalizer.py	شرط پنج ثانیه ای، عدم نهایی سازی تکراری و بافر خالی
test_label_handler.py	رفتار حروف، فاصله، حذف، NOTHING و برچسب ناشناخته
test_text_buffer.py	افزودن، حذف، فاصله، پاک سازی و متن نهایی
test_mock_predictor.py	صحت خروجی پیش بینی کننده ساختگی

تیم پروژه اجرای موفق تمام تست های موجود و بدون خطای بررسی Ruff را در محیط توسعه گزارش کرده است. تنظیمات Ruff نوت بوک ها را از تحلیل کد پایتون حذف می کند، زیرا خروجی و ساختار سلولی نوت بوک الزاماً قواعد یک ماژول تولیدی را رعایت نمی کند.

۲.۵ مدیریت خطا

موارد زیر با خطاهای اختصاصی یا پیام‌های واضح کنترل شده‌اند:

- ▶ بازنشدن دوربین یا شکست در خواندن فریم
- ▶ نبودن فایل مدل یا ناسازگاری وزن‌ها با معماری
- ▶ درخواست CUDA در سیستم فاقد CUDA
- ▶ تصویر خالی، تعداد کانال اشتباه یا آمار نرمال‌سازی نامعتبر
- ▶ خروجی مدل با شکل نامعتبر، تعداد کلاس اشتباه یا مقادیر NaN/Inf
- ▶ برچسب خالی، تکراری یا ناشناخته
- ▶ زمان‌های غیرصعودی در ماشین‌های حالت

۳.۵ قابلیت تعویض اجزا

دو کارخانه مجزا برای پیش‌بینی و گفتار وجود دارد. در مسیر پیش‌بینی، حالت mock برای توسعه و حالت torch برای مدل نهایی انتخاب می‌شود. در مسیر صوت نیز noop و pytt3 قابل انتخاب هستند. این الگو تست‌پذیری را افزایش و وابستگی مستقیم حلقه اصلی به کتابخانه‌ها را کاهش داده است.

تحلیل محدودیت‌ها و پیشنهادهای بهبود

۱.۶ شکاف دامنه میان دیتاست و وب‌کم

داده آموزشی از محیط، زاویه و نور نسبتاً همگنی برخوردار است. عملکرد 99.34% روی تقسیم داخلی همان منبع، دقت روی دوربین و فرد جدید را تضمین نمی‌کند. کلاس‌های بصری نزدیک مانند O/C/Q یا V/U/W معمولاً به زاویه انگشتان و وضوح مرزها حساس‌اند.

۲.۶ حروف و حرکات پویا

مدل ورودی تک‌فریم دارد؛ در نتیجه حرکت زمانی را مستقیماً مدل نمی‌کند. برخی حروف زبان اشاره مانند J و Z در اجرای طبیعی شامل مسیر حرکتی‌اند. مدل ممکن است یک وضعیت ثابت از این حرکت را طبقه‌بندی کند، اما برای شناخت صحیح حرکت، استفاده از دنباله فریم و معماری‌هایی مانند 3D-CNN، LSTM یا Transformer مناسب‌تر است.

راهنمای اجرا و ساختار تحویل

۱.۷ نصب

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e ".[dev,tts,model]"
```

برای نصب PyTorch سازگار با کارت گرافیک، درایور و نسخه CUDA باید از بسته مناسب سیستم استفاده شود. در صورت نبود CUDA، مقدار `device="auto"` به صورت خودکار CPU را انتخاب می کند.

۲.۷ اجرای کنترل کیفیت

```
pytest
ruff check .
```

۳.۷ اجرای سامانه

فایل وزن باید در مسیر زیر قرار داشته باشد:

`models/final_model.pth`

سپس برنامه از ریشه پروژه اجرا می شود:

```
python main.py
```

۴.۷ ساختار فایل نهایی تحویل

طبق دستور پروژه، تمام محتوا باید در یک فایل فشرده با الگوی نام‌گذاری تعیین‌شده قرار گیرد:

```
AI-Project3-StudentID1-StudentID2.zip
|-- main.py
|-- pyproject.toml
|-- models/
|   |-- final_model.pth
|-- notebooks/
|   |-- phase1&2.ipynb
|   |-- dataset_stats.pt
|-- reports/
|   |-- report.pdf
|   |-- report.tex
|-- src/sign_translator/
|-- tests/
|-- README.md
|-- LICENSE
```

نتیجه گیری

پروژه حاضر سه بخش داده، مدل و محصول بلادرنگ را در یک زنجیره یکپارچه قرار داده است. فاز اول نشان داد داده آموزشی از نظر تعداد کلاس‌ها، اندازه تصویر و حالت رنگ ساختاری منظم دارد. در فاز دوم، شبکه چهاربخشی CNN با افزایش داده و Dropout به دقت آزمون 99.34% رسید. در فاز سوم، مدل از محیط نوت‌بوک خارج و در یک برنامه واقعی مبتنی بر وب‌کم، رمزگشایی زمانی، ویرایش متن و تبدیل به گفتار به کار گرفته شد.

نقطه قوت اصلی پیاده‌سازی، جداسازی مسئولیت‌ها و توسعه افزایشی بر اساس قرارداد پیش‌بینی است. کد وب‌کم و منطق متن پیش از آماده‌شدن مدل با پیش‌بینی‌کننده ساختگی تکمیل شد و پس از تحویل وزن‌های PyTorch، تنها لایه استنتاج جایگزین گردید. استفاده از تست‌های واحد و اعتبارسنجی صریح داده‌ها نیز ریسک خطاهای خاموش در اتصال مدل را کاهش داده است.

تنظیمات نهایی سامانه

جدول ۱.آ: مقادیر پیش فرض تنظیمات اجرایی

تنظیم	مقدار
رزولوشن دوربین	1280×720
آینه سازی	فعال
اندازه ورودی مدل	200×200×3
ترتیب رنگ	RGB
چیدمان تانسور	NCHW
مدل	models/final_model.pth
دستگاه	auto (در صورت امکان، در غیر این صورت CPU)
آستانه ثبت علامت	0.80
زمان تثبیت علامت	2.0 s
زمان آزادسازی قفل	0.4 s
زمان نهایی سازی	5.0 s
موتور گفتار	pyttsx3
کلیدهای خروج	ESC و Q

قرارداد وزن‌های مدل

فایل وزن ذخیره‌شده یک OrderedDict شامل ۱۲ تنسور پارامتر است. ابعاد مشاهده‌شده در آزمون بارگذاری عبارت‌اند از:

conv1.weight	(32, 3, 3, 3)
conv1.bias	(32,)
conv2.weight	(64, 32, 3, 3)
conv2.bias	(64,)
conv3.weight	(128, 64, 3, 3)
conv3.bias	(128,)
conv4.weight	(256, 128, 3, 3)
conv4.bias	(256,)
fully_connected1.weight	(512, 36864)
fully_connected1.bias	(512,)
fully_connected2.weight	(29, 512)
fully_connected2.bias	(29,)

این ابعاد با معماری گزارش‌شده و تعداد پارامتر کل مدل سازگارند.